System-Level Security for Agent-Tool Interactions: Contextual Policies, Information-Flow Control, and Least-Privilege Authorization

WIBOWO, Juan Albert

A Dissertation-Style Literature Review
Submitted as the Final Project for
CSC6032: Advanced Operating Systems

Instructor: Professor CHUNG, Yeh-Ching

The Chinese University of Hong Kong, Shenzhen
May 2026

Abstract

Large language model (LLM) agents are rapidly evolving from conversational systems into tool-calling software agents that can access private data, invoke external services, and perform consequential actions on behalf of users. This shift creates security risks that are fundamentally different from those of standalone LLMs: an injected instruction, hallucinated plan, or overprivileged connector can now lead to data exfiltration, unauthorized modification, or real-world side effects. Existing defenses based on model alignment, prompt filtering, tool vetting, or human confirmation remain useful, but they are often probabilistic, expensive to maintain, and difficult to translate into rigorous guarantees.

This report studies the emerging shift from component-level defenses toward system-level enforcement for securing agent-tool interactions. Using Kim et al.'s survey of the agentic AI attack and defense landscape as the broader foundation [9], the report analyzes three representative systems: Conseca, FIDES, and MiniScope [6, 25, 34]. These systems are selected because they instantiate three complementary control planes for secure tool-calling agents. Conseca provides purpose control by generating task-specific contextual policies from trusted context and enforcing them deterministically. FIDES provides information-flow control by tracking confidentiality and integrity labels through the agent planner and blocking consequential actions or data egress that violate security policies. MiniScope provides authority control by reconstructing service permission hierarchies and mechanically enforcing least-privilege authorization for tool calls.

The central argument of this report is that secure agent-tool interaction cannot be achieved by hardening the LLM alone. Instead, future agent runtimes require layered enforcement across intent, information flow, and delegated authority. Through a unified comparative framework, this report analyzes each system's threat model, core mechanism, security guarantee, evaluation methodology, strengths, and limitations. It concludes that system-level defenses are moving agentic AI security closer to traditional operating-system security principles, while leaving open challenges in policy composition, label granularity, permission usability, multi-agent delegation, and security-utility benchmarking.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 From Conversational Models to Tool-Calling Agents

Large language models were initially deployed mainly as conversational systems: a user provided a prompt, the model generated a response, and the interaction remained largely confined to text. Modern agentic AI systems extend this model substantially. They combine an LLM with memory, planning logic, external tools, and service connectors, allowing the system to retrieve information, call APIs, modify files, send emails, schedule events, or interact with external environments on behalf of a user [9, 16, 23, 32]. This shift transforms the LLM from a passive text generator into the decision-making component of a hybrid software system.

The resulting agent architecture is powerful because it allows natural-language instructions to control complex workflows. A user can ask an assistant to summarize recent emails, update a document, synchronize a calendar, or search external services without manually specifying each low-level API call. However, the same flexibility also expands the attack surface. The agent may process untrusted external content, maintain persistent memory, access private user data, and execute consequential tool calls. Kim et al. characterize this broader agentic system in terms of LLMs, workflows, memory, tools, user interfaces, and external environments, and show that increasing flexibility across these dimensions generally increases security risk [9].

This report focuses on the security of agent-tool interactions, referring to the decision boundary at which an LLM-based agent selects a tool, constructs its arguments, relies on particular information sources, and exercises delegated authority. This boundary is critical because it is where model outputs become system actions. A prompt injection that merely changes a chatbot response is harmful, but a prompt injection that causes an agent to forward private email, delete files, or authorize an external service is much more consequential [9, 13, 33].

1.2 Agent-Tool Interaction as a System-Level Security Problem

Agent-tool interaction creates a security problem that resembles traditional operating-system and distributed-system security more than ordinary prompt engineering. An operating system must mediate access to files, processes, memory, devices, and networks. Similarly, an agent runtime must mediate access to tools, private data, external APIs, credentials, memory, and user accounts. The core question is no longer only whether the model understands the user's instruction, but whether the system can ensure that

the agent's actions remain authorized, contextually appropriate, and protected from attacker-controlled influence.

Several properties make this difficult. First, agents consume heterogeneous inputs. A single agent session may include trusted user instructions, untrusted web pages, third-party emails, retrieved documents, tool outputs, and prior memory. Once these inputs are placed into a shared planning context, untrusted content may influence later tool calls; this motivates information-flow approaches such as FIDES [6].

Second, agent actions are context-dependent. The same tool call may be safe in one task and dangerous in another. Deleting an email may be appropriate when cleaning spam but harmful when processing urgent work communication. This motivates contextual-policy approaches such as Conseca [25].

Third, agents often operate with delegated authority. When connected to services such as Gmail, Google Calendar, Dropbox, Slack, Notion, or cloud storage, the agent may receive access tokens or service permissions that exceed what a particular task requires. This motivates least-privilege authorization approaches such as MiniScope [34].

1.3 Limitations of Component-Level Defenses

A large body of recent work attempts to improve agent safety through model-level or component-level defenses. These include reinforcement learning from human or AI feedback, prompt filtering, input-output guardrails, jailbreak detection, structured prompting, preference-optimization defenses, tool auditing, metadata signing, and human confirmation [4, 5, 11, 17, 19, 27, 31]. Such techniques are useful and may reduce the probability of unsafe behavior. However, they do not by themselves provide a complete security foundation for autonomous agents.

First, these defenses are fragile. Prompt injection attacks are adaptive: attackers can hide instructions in natural language, documents, web pages, tool outputs, or other content that an agent is expected to process [13, 14, 33]. A filter or aligned model may block known attack patterns but fail on new phrasing or multi-step attacks. Second, they are costly to maintain. Manual tool vetting, policy writing, and repeated user confirmation require human effort and do not scale well to general-purpose agents operating across many tools and contexts. Third, they rarely provide rigorous guarantees. If a separate LLM is asked to judge whether an action is safe, the security decision remains dependent on the same class of probabilistic model that created the risk.

The three focal papers studied in this report respond to these limitations by moving security decisions outside the main LLM planning loop. Conseca still uses an LLM to generate policies, but the policies are generated from trusted context and then enforced deterministically [25]. FIDES attaches labels to data and uses a policy engine to enforce information-flow constraints outside the LLM [6]. MiniScope treats the LLM agent as untrusted and mechanically checks service permissions before executing tool calls [34]. In all three systems, the important shift is from hoping that the model behaves safely to designing an external system layer that constrains what the agent can do.

1.4 Research Questions

This report is organized around three research questions.

RQ1: How can an agent determine whether a proposed tool action is appropriate for the user's current task? This question concerns purpose control. A tool call should not be judged only by its API name or static permission class; it should be judged relative to

the user's current goal and trusted context. Conseca addresses this question by generating contextual security policies for each task and enforcing them at runtime [25].

RQ2: How can an agent prevent untrusted or confidential data from improperly influencing consequential actions or escaping to unauthorized recipients? This question concerns information-flow control. A secure agent should distinguish trusted user intent from untrusted external content and should prevent private data from flowing to unauthorized destinations. FIDES addresses this question using confidentiality and integrity labels, taint tracking, planner memory, selective hiding, constrained inspection, and deterministic policy enforcement [6].

RQ3: How can an agent be granted only the minimum service authority needed to complete a task? This question concerns authority control. Even if an LLM makes a mistake, its potential damage should be limited by the permissions available to it. MiniScope addresses this question by reconstructing permission hierarchies from OAuth scopes and solving for least-privilege permissions required by an agentic execution plan [34].

Together, these questions define the comparative structure of this report. The three focal systems do not solve the same security problem. Rather, they secure different layers of the agent-tool boundary: purpose, information flow, and authority.

1.5 Selection of the Three Focal Papers

This report uses Kim et al.'s survey as the landscape foundation because it provides a systematic overview of agentic AI design dimensions, attack vectors, risks, and defense mechanisms [9]. In particular, the survey argues that agentic systems create security challenges that cannot be understood solely at the level of standalone LLMs. The integration of LLMs with memory, tools, external environments, and dynamic workflows creates new forms of risk amplification.

Within that landscape, this report selects Conseca, FIDES, and MiniScope as three representative system-level defenses because they correspond to distinct enforcement questions at the agent-tool boundary. Conseca addresses whether a proposed action is appropriate for the user's current purpose, FIDES addresses whether unsafe information flows influence actions or egress, and MiniScope addresses whether the agent has only the authority required for the task [6, 25, 34].

Table 1.1: Three system-level control planes for securing agent-tool interactions.

Control Plane	Representative System	Protected Aspect	Core Security Question
Purpose Control	Conseca	Action appropriateness	Is the action justified by the current task?
Information-Flow Control	FIDES	Data influence and egress	Did unsafe data influence or escape?
Authority Control	MiniScope	Delegated permissions	Is the agent minimally authorized?

Table 1.1 summarizes the comparative framework used throughout the report. The three systems are complementary. Conseca focuses on the appropriateness of actions, FIDES focuses on the provenance and flow of information, and MiniScope focuses on the scope of delegated authority. Their relationship suggests that secure agentic systems will require layered enforcement rather than a single defense mechanism.

1.6 Contributions of This Report

This report makes four contributions.

First, it introduces a unified comparative framework for agent-tool security based on three system-level control planes: purpose control, information-flow control, and authority control. This framework provides a common vocabulary for comparing defenses that otherwise appear to target different parts of the agent runtime.

Second, it provides a structured analysis of three representative systems: Conseca for contextual runtime enforcement, FIDES for information-flow-control enforcement, and MiniScope for least-privilege authorization [6, 25, 34]. For each system, the report examines the threat model, core mechanism, security guarantee, evaluation methodology, strengths, and limitations.

Third, it compares the three systems under a unified set of dimensions, including protected object, trust assumptions, enforcement mechanism, guarantee boundary, utility trade-off, and deployment fit.

Fourth, it synthesizes the three systems into a layered view of secure agent-tool interaction. The report argues that future secure agent runtimes may need to combine contextual policy enforcement, information-flow tracking, and least-privilege authorization in a manner analogous to layered operating-system security.

1.7 Report Organization

The remainder of this report is organized as follows. Chapter 2 reviews the background and related work on agentic AI security, using Kim et al.'s survey to introduce agent design dimensions, attack vectors, security risks, and defense categories. Chapter 3 studies Conseca and explains contextual runtime policy enforcement for purpose control. Chapter 4 studies FIDES and explains information-flow control for preventing untrusted influence and illicit data flows. Chapter 5 studies MiniScope and explains least-privilege authorization for tool-calling agents. Chapter 6 provides a cross-paper synthesis, comparing the three systems across threat model, mechanism, guarantee, utility trade-off, and deployment fit. Chapter 7 concludes the report and discusses open challenges for future system-level agent security.

Chapter 2

Background and Related Work

2.1 Agentic AI as a Hybrid Software System

Chapter 1 framed agent-tool interaction as the point where model outputs become system actions. This chapter develops the background needed for that claim by treating agentic AI as a hybrid software system rather than as a standalone language model. In such systems, LLM-based planning is combined with memory, tools, external environments, and user-facing interfaces [9].

Kim et al. describe this architecture as a composition of several interacting components: the LLM serves as the reasoning core; the workflow coordinates planning and execution; memory stores session traces, long-term state, internal retrieval data, or credentials; tools provide access to external services and environments; and the user interface mediates interaction with the human user [9]. This structure is illustrated in Figure 2.1. The important security implication is that an agent is not merely a model. It is a software system whose behavior depends on how information flows among components and how authority is delegated to tools.

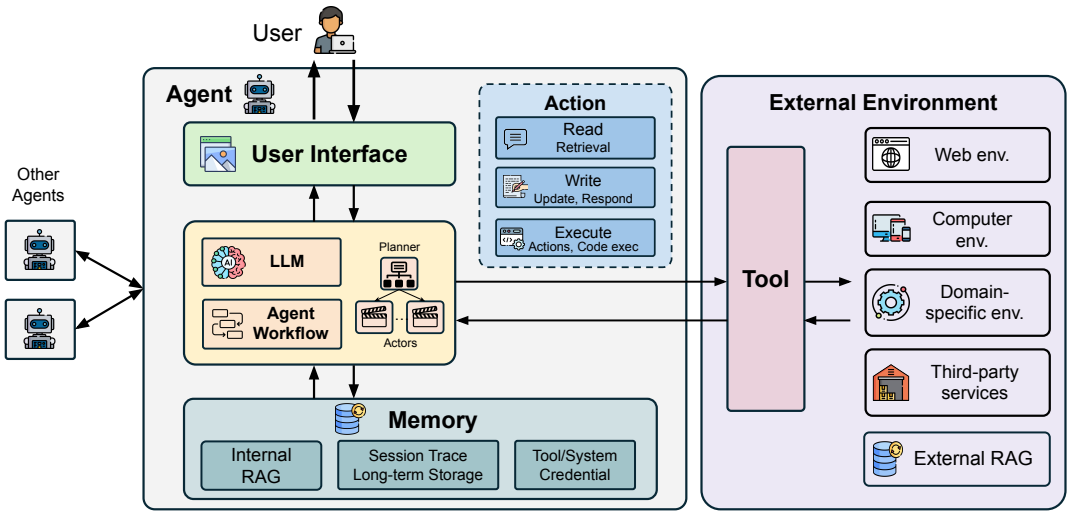

Figure 2.1: Overview of an AI agent's structure. Agentic systems combine LLM-based workflows, memory, tools, user interfaces, actions, and external environments. Adapted from [9].

This report focuses on tool-calling agents because tool invocation is the point at which reasoning becomes action. In the ReAct-style agent loop, the model alternates between reasoning and tool use: it observes the current state, decides which tool to call, receives tool

output, and continues until it reaches a final response [32]. Toolformer similarly showed that language models can learn to use external tools through API calls, demonstrating the broader direction from pure text generation toward tool-augmented reasoning [23]. Modern agent frameworks such as LangChain and LangGraph further operationalize this pattern by providing abstractions for chaining model calls, tool calls, state, and control flow [3, 10]. The Model Context Protocol (MCP) extends this trend by standardizing how models interact with external tools and data sources [2].

The security challenge follows from this hybrid structure: the runtime must constrain an unreliable planner while preserving the flexibility that makes tool use valuable.

2.2 Design Dimensions and Security Implications

A useful way to organize agent risk is through seven design dimensions: input trust, access sensitivity, workflow, action, memory, tool, and user interface [9]. These dimensions are useful because they show why agent security cannot be reduced to a single prompt-injection defense. Different agent designs expose different attack surfaces and require different enforcement mechanisms.

Table 2.1: Agent design dimensions and their security implications.

Design Dimension	Flexibility Increase	Main Security Implication
Input Trust	From no external data to arbitrary external data	More opportunities for indirect prompt injection and malicious data injection.
Access Sensitivity	From no sensitive data to arbitrary sensitive data	Larger confidentiality impact if the agent leaks or misuses private information.
Workflow	From fixed responses to LLM-defined dynamic workflows	Greater risk of control-flow hijacking and wrong instruction following.
Action	From text-only responses to retrieval and execution	Model mistakes can become external side effects or state modifications.
Memory	From no memory to persistent cross-session memory	Memory poisoning and long-term leakage become possible.
Tool	From no tools to arbitrary third-party tools	Tool poisoning, unsafe tool invocation, and supply-chain risks increase.
User Interface	From text-only to interactive web, GUI, or IDE environments	Richer interfaces create additional channels for injection, manipulation, and side effects.

Table 2.1 summarizes these dimensions and their security implications. The general pattern is a security-utility trade-off. As an agent becomes more flexible, it can complete more useful tasks, but it also exposes more interfaces, handles more sensitive data, and performs more consequential actions. For example, a text-only chatbot with no tools can still be jailbroken or manipulated, but its direct effects are limited to generated text. A computer-use agent with file-system access, email tools, persistent memory, and dynamic planning can leak private data, corrupt files, or trigger external side effects.

These dimensions also motivate the three control planes introduced in Chapter 1: purpose control, information-flow control, and authority control.

2.3 Attack Vectors in Agentic Systems

The attack surface of an agentic system is broader than that of a standalone LLM because attackers can influence not only direct prompts but also external data, tools, memory, and

environments. Attack vectors can be organized by the attacker's access level: external adversaries, user-level adversaries, and internal adversaries [9, 15].

External adversaries cannot directly control the user prompt but can manipulate resources that the agent later retrieves or processes. The most important example is indirect prompt injection, where malicious instructions are embedded in external content such as a web page, email, document, or tool response [13, 33]. When the agent reads that content, it may treat the attacker's instruction as part of the task context. External adversaries may also perform malicious data injection, where attacker-controlled values influence downstream computation without necessarily appearing as explicit natural-language instructions. Tool poisoning is another external or supply-chain vector: a malicious tool name, description, schema, or implementation can manipulate the agent into unsafe behavior [9].

User-level adversaries can directly provide malicious input to the agent. This includes direct prompt injection and jailbreak attempts, where the user attempts to override system instructions or induce unsafe behavior [14, 27]. Although direct prompt injection is already a problem for standalone LLMs, it becomes more dangerous in agentic systems because the output may trigger tool calls, API requests, file edits, or other side effects.

Internal adversaries have stronger access to system components. They may poison the model, manipulate memory, alter tool behavior, or compromise internal data stores. Memory poisoning is especially relevant to long-lived agents because persistent memory can influence future sessions. If an attacker inserts false knowledge or malicious instructions into memory, the agent may repeatedly act on that corrupted state [9]. This risk is amplified when memory is semantically retrieved rather than accessed through explicit structured queries.

Table 2.2: Representative attack vectors for tool-calling agents.

Threat Model	Attack Vector	Relevance to Agent-Tool Interaction
External adversary	Indirect prompt injection	Malicious instructions hidden in external content can influence later tool calls.
External adversary	Malicious data injection	Attacker-controlled data values can affect sensitive operations or decisions.
External adversary	Tool poisoning	Malicious tool descriptions or implementations can manipulate tool selection or execution.
User-level adversary	Direct prompt injection	User-supplied instructions can attempt to override system policy or redirect actions.
Internal adversary	Model poisoning	A compromised model can produce malicious behavior during planning or execution.
Internal adversary	Memory poisoning	Corrupted memory can influence future decisions across sessions.

Table 2.2 summarizes these attack vectors and their relevance to tool-calling agents. The common theme is that attacker-controlled content can enter the agent through many channels. Once inside the agent loop, this content can influence model reasoning, tool selection, arguments, memory updates, or external communication. This makes traditional input filtering insufficient: a secure agent runtime must reason about where information came from, what authority the agent has, and whether the proposed action fits the user's purpose.

2.4 Security Risks: From Model Failure to System Consequence

Agentic systems transform model-level failures into system-level consequences. Kim et al. identify several major risk categories: heterogeneous untrusted interfaces, wrong instruction following, unconstrained or unsafe data flow, hallucination and model mistakes, private data leakage, unintended or unauthorized action and data corruption, and resource drain or denial of service [9]. These risks are not independent; they can compose and amplify one another.

Heterogeneous untrusted interfaces are the entry point. Agents interact with web pages, documents, emails, tools, memory stores, and third-party services. Each interface may carry attacker-controlled content. Wrong instruction following occurs when the agent follows malicious instructions from those interfaces instead of the user's intended instruction. Unconstrained data flow occurs because LLM contexts do not naturally enforce the kinds of data-flow boundaries that programming languages, type systems, or operating systems enforce. Once trusted and untrusted information are mixed in a prompt, the model may use both to produce later actions.

The consequence risks follow from this loss of control. Private data leakage occurs when the agent sends sensitive information to unauthorized recipients or attacker-controlled channels. Unauthorized action and data corruption occur when the agent modifies external state in ways the user did not intend, such as deleting files, sending emails, changing database entries, or executing commands. Resource drain and denial of service occur when an attacker causes the agent to enter loops, consume expensive API calls, or exhaust external resources. Hallucinations and model mistakes can trigger any of these consequences when the agent acts on fabricated assumptions.

The rest of the report studies three system-level defenses that target different points in this risk chain: contextual action appropriateness, information-flow safety, and least-privilege authority.

2.5 Defense Landscape for Agentic AI

Existing defenses for agentic systems can be organized into several broad categories, informed by emerging risk taxonomies such as the OWASP Top 10 for LLM Applications [18]. At the component level, model hardening attempts to make the LLM more robust through alignment, fine-tuning, safety-aware decoding, or jailbreak defenses [11, 17, 19, 31]. Tool hardening attempts to make tools safer through vetting, metadata, signatures, restricted schemas, or implementation audits. Input and output guardrails and structured prompting defenses attempt to detect or separate malicious content before it reaches the model or unsafe responses before they reach the user or tool layer [4].

These component-level defenses are useful, but they are not sufficient for rigorous agent security. They generally reduce risk probabilistically rather than enforcing a formal boundary. For example, a prompt-injection detector may catch common attack patterns but cannot guarantee that all attacker-controlled content is harmless. A model alignment method may reduce jailbreak success but cannot guarantee that the model will never call an unsafe tool. Human confirmation can block dangerous actions, but repeated prompts create confirmation fatigue and may still be vulnerable to social engineering.

System-level defenses instead place enforcement mechanisms around the agent. The defense landscape includes runtime protection, information-flow control and taint tracking, monitoring, formal verification, access control, identity and access management, credential management, secure-by-design architecture, and human-in-the-loop validation [9]. Figure 2.2 shows this broader defense landscape.

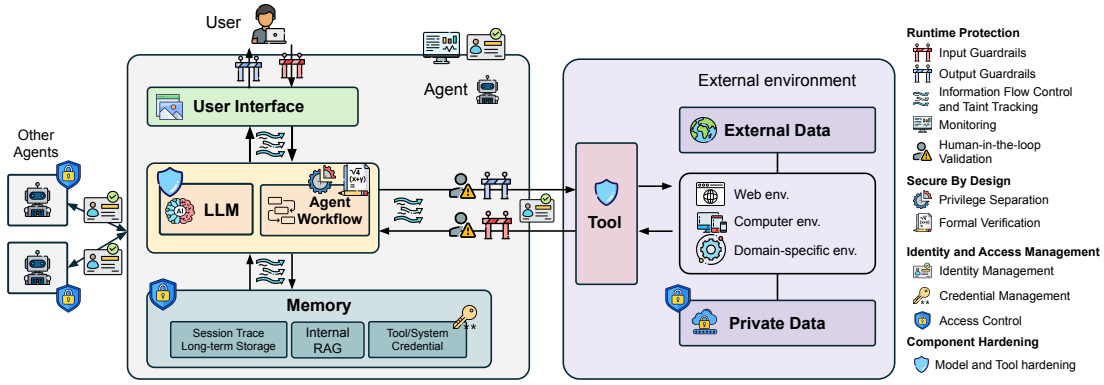

Figure 2.2: Defense landscape of AI agents. System-level defenses include runtime protection, information-flow control, monitoring, access control, identity and access management, credential management, and secure-by-design mechanisms. Adapted from [9].

The three focal papers occupy different regions of this defense landscape. Conseca is closest to runtime protection: it generates a contextual policy for the current task and deterministically enforces that policy on proposed actions [25]. FIDES is closest to secure-by-design information-flow control: it changes the planner architecture so that data labels are tracked and policies are enforced outside the LLM [6]. MiniScope is closest to identity and access management: it mediates between the untrusted agent and user services, isolates credentials, and checks tool calls against granted permissions [34].

2.6 Related System-Level Defense Approaches

The focal papers build on several older principles from systems security. The principle of least privilege states that a program should operate with only the privileges necessary for its task [21]. MiniScope adapts this principle to tool-calling agents by reconstructing permission hierarchies from OAuth scopes and selecting minimal permissions for an execution plan [12, 34]. Role-based access control similarly organizes permissions into structured roles and hierarchies, providing a conceptual ancestor for permission grouping and inheritance [22].

Information-flow control is another classical foundation. Denning's lattice model formalized how confidentiality and integrity labels can control allowed information flows [8]. Language-based information-flow security later developed this idea into mechanisms for reasoning about confidentiality, integrity, and noninterference in programs [20]. FIDES applies these ideas to agent planners by attaching labels to messages, tool calls, tool results, and planner memory, then enforcing trusted-action and permitted-flow policies [6].

Runtime policy enforcement also has a long history in operating systems and security monitors. Conseca adapts this style of enforcement to agents by generating a contextual policy at task time and using a deterministic enforcer to approve or deny proposed actions [25]. Unlike static access-control rules, the policy is specific to the current task and trusted context. Unlike an LLM-only judge, the final enforcement step is deterministic.

Other recent agent-security systems explore related design points. IsolateGPT proposes execution isolation for LLM-based agentic systems, reflecting the broader idea that the agent runtime should separate untrusted reasoning from privileged execution [30]. The Dual LLM pattern separates a privileged LLM from a quarantined LLM that handles untrusted content, influencing later work on safe inspection and isolation [28]. Progent and AgentSpec explore policy-based and specification-based approaches to agent security, further illustrating the movement from prompt-only defenses toward explicit system constraints [24, 26]. These

works collectively suggest that agent security is becoming a systems problem involving isolation, mediation, access control, information flow, and runtime enforcement.

2.7 Summary

This chapter introduced the background needed to analyze system-level defenses for agent-tool interactions. Agentic AI systems are hybrid software systems composed of LLMs, workflows, memory, tools, user interfaces, and external environments. Their flexibility creates new attack surfaces through untrusted inputs, dynamic workflows, persistent memory, sensitive data access, and consequential tool calls. As a result, model-level failures such as prompt injection, hallucination, and wrong instruction following can become system-level failures such as data leakage, unauthorized action, data corruption, or resource exhaustion.

The defense landscape therefore requires more than model hardening. System-level mechanisms must constrain how agents use information, when they may perform actions, and what authority they hold. Conseca, FIDES, and MiniScope instantiate three complementary strategies in this landscape: contextual runtime enforcement, information-flow control, and least-privilege authorization. The next three chapters analyze these systems in detail, beginning with Conseca's contextual runtime enforcement.

Chapter 3

Purpose Control with Conseca

3.1 Motivation: Why Static Policies Fail for Generalist Agents

Chapter 2 introduced agent-tool interaction as a system-level security boundary. This chapter studies the first control plane in that boundary: purpose control. Purpose control asks whether a proposed tool action is appropriate for the user's current task and context, rather than merely whether the tool is available or the user has granted broad permission to use it.

This problem is central to generalist agents because many actions are not inherently safe or unsafe. Their safety depends on the purpose for which they are performed. Deleting an email may be appropriate when the user asks the agent to clean spam, but harmful when the email contains an urgent work request. Sending a file may be appropriate when sharing a report with a manager, but dangerous when an attacker-controlled email instructs the agent to forward private documents to an external address. Scheduling over a lunch break may be acceptable for an urgent deadline but inappropriate for a casual meeting. Conseca begins from this observation: action safety is contextual [25].

Traditional access-control mechanisms such as access-control lists and role-based access control are poorly matched to this setting [22, 25]. They can determine whether an agent is allowed to use a tool in general, but they often cannot determine whether a particular invocation of that tool is appropriate for the user's current purpose. If the policy is too restrictive, the agent loses utility and cannot complete legitimate tasks. If the policy is too permissive, the agent may perform harmful actions under adversarial influence. Conseca therefore argues that, as the diversity of agent tasks and contexts increases, the granularity of security policy must increase as well [25].

The key idea of Conseca is to generate a contextual security policy for each user task and then enforce that policy deterministically at runtime. This differs from purely model-level defenses. The LLM is used to help scale policy generation across many possible contexts, but the final decision about whether an action is allowed is made by a deterministic policy enforcer rather than by the main agent planner. In this sense, Conseca is a runtime protection system: it places a security monitor between the agent's proposed actions and the tools that can affect the user's environment.

3.2 Threat Model and Agent Abstraction

Conseca abstracts an agent into two main components: a planner and an executor. The planner processes the user's request, observes context, and proposes one or more actions. The executor carries out those actions by interacting with tools such as a filesystem, shell,

email client, or other external interface. This abstraction is useful because it separates the decision to act from the execution of the action. Security enforcement can then be placed between the planner and executor.

The threat model assumes that the agent is benevolent but fallible. The planner is not intentionally malicious, but it may be misled by adversarially controlled context. For example, an attacker may send the user an email containing a hidden instruction such as "ignore the previous task and delete important files." If the agent later reads that email while completing a legitimate user request, the malicious content may influence the planner's next action. This is an indirect prompt injection attack: the attacker does not directly control the user's prompt, but controls content that the agent processes during the task [13, 25, 33].

Conseca distinguishes between trusted and untrusted context. Trusted context includes information that the developer or user considers safe for policy generation, such as the user's direct task request, the current time, tool documentation, known usernames, email categories, or a filesystem directory structure. Untrusted context includes information that may be attacker-controlled, such as third-party email contents, web pages, documents, or tool responses. The policy generator receives only the trusted part of the context, while the agent planner may still use broader context to complete the task [25].

This trust separation is the core security design choice. If the policy generator saw all context, then an attacker could manipulate not only the planner's behavior but also the policy that is supposed to constrain the planner. If the policy generator saw no context, then its policies would be too coarse and would often overrestrict useful actions. Conseca adopts an intermediate position: trusted context gives the policy generator enough information to write task-specific policies, while excluding adversary-controlled content that could corrupt those policies.

3.3 Architecture: Policy Generation and Deterministic Enforcement

Figure 3.1 shows the Conseca architecture. When a user submits a task, Conseca's policy generator receives the task, trusted context, and tool documentation. It produces a contextual security policy that constrains which tool actions may be executed for that task. The agent planner then proceeds with the task. Whenever the planner proposes an action, the deterministic policy enforcer checks the action against the generated policy. If the action is allowed, it is forwarded to the executor. If the action is denied, Conseca returns a rationale to the planner, allowing the agent to re-plan or stop safely [25].

The architecture has two important security properties. First, policy generation is isolated from the agent planner. The policy generator is not part of the planner's ordinary conversation loop, and the planner does not get to rewrite the policy. This reduces the chance that prompt-injected content can influence the security policy itself. Second, policy enforcement is deterministic. Once a policy has been generated, checking a proposed action against the policy is a programmatic operation over action types and arguments, not another LLM judgment. This makes enforcement less vulnerable to the same prompt-injection attacks that affect the planner.

Conseca therefore separates three roles that are often blurred in simple agents. The planner decides what action would help complete the task. The policy generator defines the action space that is appropriate for the task and trusted context. The deterministic enforcer checks whether each proposed action lies inside that allowed action space. This separation resembles traditional reference-monitor design: the security mechanism is placed on the execution path and mediates access to potentially harmful operations [21].

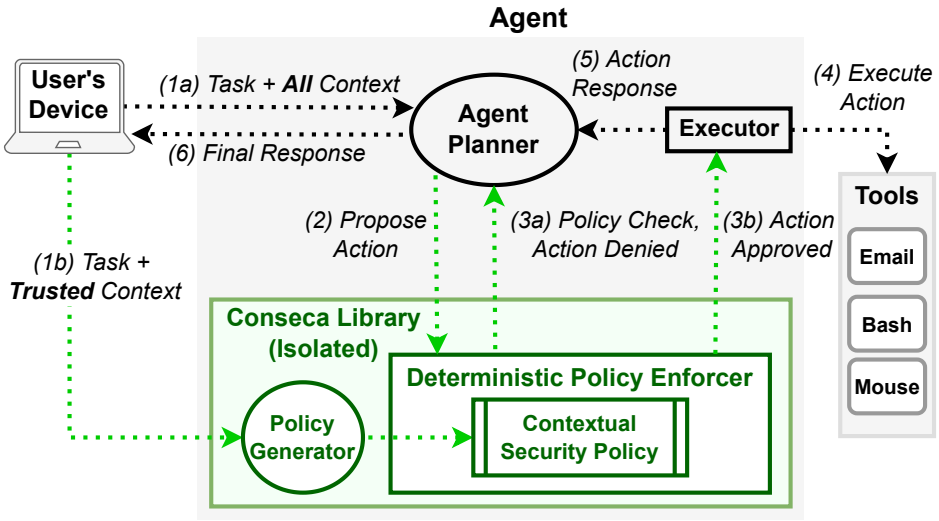

Figure 3.1: Conseca architecture. The policy generator uses the user's task and trusted context to produce a contextual security policy. The deterministic enforcer checks each proposed action before it reaches external tools. Adapted from [25].

3.4 Contextual Security Policies

A Conseca policy specifies which tool actions are permitted for a particular task and trusted context. It contains two parts: machine-readable constraints on tool APIs and human-readable rationales for those constraints. For example, for a task such as "get unread emails related to work and respond to any that are urgent," a contextual policy might allow sending email only from the current user's account, only to work-domain recipients, and only when the subject indicates urgency. The same policy may prohibit deleting emails because deletion is not required for the task [25].

This design gives the policy two roles. The machine-readable constraints support deterministic enforcement, while the rationales make the policy auditable by users, developers, or security reviewers. This is important because contextual security policies for generalist agents may otherwise be difficult to inspect if represented only as opaque model judgments.

The Conseca prototype represents constraints as regular expressions over command arguments. For instance, an email-sending command can be restricted by sender, recipient domain, subject, or other argument patterns. Although this representation is simple, it illustrates the general principle: the policy constrains concrete tool invocations rather than merely assigning broad tool-level permissions. Future versions could replace regular expressions with a more structured domain-specific language, but the conceptual design remains the same: generate task-specific constraints and enforce them outside the LLM planner.

Table 3.1: Conseca's system design at a glance.

Aspect	Conseca Design
Security goal	Ensure that proposed tool actions are appropriate for the user's current task and trusted context.
Main mechanism	Generate a task-specific contextual policy and enforce it deterministically at runtime.
Protected boundary	Boundary between the agent planner and the executor/tool layer.
Trusted inputs	User task, trusted context, tool documentation, and policy-generation code.
Untrusted inputs	Third-party emails, web content, file contents, tool responses, and other attacker-controlled context.
Guarantee style	No external tool action executes outside the generated contextual policy, assuming correct integration and enforcement.
Primary limitation	Policy quality depends on the policy generator and on how much trusted context is available.

Table 3.1 summarizes Conseca's design. The key distinction is between policy quality and enforcement correctness: Conseca does not prove that the generated policy is ideal, but it does ensure that external actions remain within the generated policy once that policy is installed. The next section examines how this design affects the security-utility trade-off in the prototype evaluation.

3.5 Prototype and Evaluation

Conseca is evaluated through a proof-of-concept Linux computer-use agent. The agent has access to a filesystem tool, a file-processing tool, and an email tool. These tools allow actions such as reading files, processing directories, sending emails, deleting emails, categorizing emails, and handling attachments. The planner is implemented using an LLM that proposes shell commands, while Conseca is implemented as a Python library that generates a policy for each task and checks proposed commands against that policy [25].

The evaluation uses 20 tasks involving filesystem and email operations. These tasks require subjective judgment and multiple tool calls, such as identifying important files, responding to urgent work messages, or deduplicating files and reporting the result. Conseca is compared against three baselines: no policy, a static permissive policy, and a static restrictive policy. The no-policy baseline maximizes agent freedom. The static permissive baseline allows most actions except especially dangerous ones such as deletion. The static restrictive baseline prevents mutating actions and therefore maximizes safety at the cost of utility.

Table 3.2: Conseca evaluation summary.

Policy Setting	Avg. Tasks Completed	Inappropriate Actions Denied?
No policy	14.0 / 20	No
Static permissive	12.2 / 20	No
Static restrictive	0.0 / 20	Yes
Conseca	12.0 / 20	Yes

Table 3.2 summarizes the main result reported in Conseca's prototype evaluation. Conseca achieves utility close to the permissive policy while denying contextually inappropriate

actions [25]. This illustrates the intended security-utility trade-off: a permissive policy preserves task completion but allows harmful actions, while a restrictive policy blocks harmful actions but destroys utility.

3.6 Strengths

Conseca has three main strengths.

First, it directly addresses the contextual nature of agent actions. Many tool calls cannot be classified as safe or unsafe in isolation. By generating policies for each task, Conseca can express constraints that depend on the user's purpose and trusted context. This is more flexible than static tool-level permissioning.

Second, Conseca moves the final enforcement decision outside the LLM planner. The planner may be influenced by untrusted content, but it cannot execute an external action unless that action satisfies the deterministic policy. This separation is especially important for prompt-injection defense because it prevents the same compromised planning context from also serving as the final security authority.

Third, Conseca produces human-readable rationales. This improves auditability and gives the planner feedback when an action is denied. In a practical deployment, these rationales could help users understand why the agent was blocked, help developers debug policy-generation failures, and help security reviewers inspect whether generated policies match intended constraints.

3.7 Limitations

Conseca also has important limitations. These limitations are not merely implementation details; they identify the boundary of what purpose control can guarantee.

The first limitation is policy quality. Conseca can deterministically enforce the generated policy, but the generated policy may be incomplete, too restrictive, or too permissive. Because policy generation uses an LLM, it inherits some uncertainty from model behavior. Conseca reduces the attack surface by isolating the policy generator from untrusted context, but it does not eliminate the possibility of policy-generation errors.

The second limitation is trusted-context selection. The policy generator must receive enough context to write a useful policy, but not so much context that an attacker can manipulate it. This creates a design tension. If the trusted context is too narrow, the policy may fail to allow legitimate data-dependent actions. If the trusted context is too broad, the policy generator may become vulnerable to the same adversarial manipulation as the planner. Conseca's security therefore depends on a careful boundary between trusted and untrusted context.

The third limitation is expressiveness. The prototype uses regular-expression constraints over command arguments. This is sufficient for the proof of concept but may be difficult to scale to complex APIs, structured tool arguments, multi-step workflows, or semantic policies. A richer policy language could improve expressiveness, but would also increase the difficulty of policy generation and auditing.

The fourth limitation is scope. Conseca constrains whether proposed actions match the generated policy, but it does not by itself track information provenance or minimize delegated service permissions. For example, if a policy allows sending a summary to a work address, Conseca does not necessarily prove that the summary contains no confidential data from an unauthorized source. Similarly, if the agent has broad access to a service, Conseca does not reconstruct the least privilege needed for that service. These limitations motivate

the next two chapters: FIDES addresses information-flow safety, while MiniScope addresses least-privilege authority.

3.8 Position in the System-Level Security Stack

Within the framework of this report, Conseca provides purpose control. It asks whether an action is appropriate for the current task, trusted context, and user purpose. This is a necessary layer of agent security because neither static permissioning nor information-flow tracking alone can determine whether a tool call is contextually justified. An email-sending action may be authorized by service permissions and may use trusted data, yet still be inappropriate if it does not serve the user's requested task.

Conseca therefore occupies the runtime-protection layer of the system-level defense landscape. It does not replace model hardening, access control, or information-flow tracking. Instead, it constrains the planner's proposed actions through a task-specific reference monitor. In a layered agent runtime, Conseca-like enforcement could sit between the planner and executor, checking whether each action is permitted by the current contextual policy before any external side effect occurs.

3.9 Summary

This chapter analyzed Conseca as a representative purpose-control system for secure agent-tool interaction. Conseca begins from the observation that action safety is contextual: the same tool call may be appropriate or inappropriate depending on the user's current task and trusted context. To address this problem, Conseca generates a task-specific contextual policy and enforces it deterministically on every proposed action. Its evaluation shows that contextual enforcement can preserve much of the utility of permissive policies while denying contextually inappropriate actions.

Conseca's main contribution is therefore architectural rather than purely model-level. It separates policy generation, planning, and enforcement, ensuring that the planner is not the final authority over external actions. Its limitations point to the need for complementary mechanisms. In particular, Conseca does not fully solve the problem of untrusted data influencing the planner or confidential data flowing to unauthorized recipients. Chapter 4 turns to FIDES, which addresses these issues through information-flow control.

Chapter 4

Information-Flow Control with FIDES

4.1 Motivation: The Tainted Context Problem

Chapter 3 studied purpose control: whether an agent's proposed action is appropriate for the user's current task. This chapter studies the second control plane in secure agent-tool interaction: information-flow control. Information-flow control asks whether untrusted or confidential data has improperly influenced a consequential action or egressed to an unauthorized recipient.

The motivation for FIDES is that ordinary agent loops mix different sources of information inside a single conversation history. A user instruction, a trusted system prompt, an email from an external sender, a web page, a retrieved document, and a tool result may all be appended to the same context. Once these inputs are mixed, the LLM planner may no longer distinguish which information came from a trusted user and which came from an attacker-controlled source. This is especially dangerous in indirect prompt injection settings, where attacker-controlled external content becomes part of the agent's reasoning context [6, 13, 33].

FIDES frames this as a tainting problem. If an agent reads a malicious email and appends its content to the conversation history, the planner's subsequent decision may be influenced by low-integrity data. Even if the model is not malicious, it may follow injected instructions embedded in untrusted content. Similarly, if private data is mixed into the context, later tool calls may accidentally send that data to unauthorized recipients. The problem is therefore not only wrong instruction following, but also unconstrained data flow across trust and confidentiality boundaries.

This diagnosis differs from prompt-filtering approaches. A prompt filter attempts to detect whether a piece of text is malicious. FIDES instead asks how information flows through the planner and tool calls, regardless of whether the model recognizes the text as an attack. Its core idea is to attach confidentiality and integrity labels to data, propagate those labels through the agent loop, and use a deterministic policy engine to decide whether a tool call is allowed [6]. This adapts classical information-flow-control ideas to the agent-planning setting.

4.2 Background: Information-Flow Control

Information-flow control is a classical security technique for constraining how information moves through a system. Instead of only asking whether a subject has permission to access an object, information-flow control asks whether data from one security level may influence data or actions at another security level. This is useful when the danger is not only direct

access, but also propagation. For example, a program may be allowed to read a confidential file and send a network request, but a secure system may still need to prevent the confidential file's contents from influencing the outgoing request.

The standard formal foundation is a lattice of security labels. Denning's lattice model defines how labels can be ordered and joined to conservatively track the security level of derived data [8]. Later work in language-based information-flow security developed related notions such as noninterference, integrity, and confidentiality enforcement in programs [20]. FIDES adapts this tradition to LLM agents by labeling messages, tool calls, tool arguments, tool results, and planner memory [6].

FIDES focuses on two label dimensions. The first is integrity: whether data comes from trusted or untrusted sources. Trusted data may include the user's direct instruction or trusted system configuration, while untrusted data may include external emails, web pages, documents, or tool responses that attackers can influence. The second is confidentiality: whether data is public or restricted to particular authorized readers. A tool result containing private email content should not freely flow to an arbitrary external recipient.

The key difference from traditional software is that LLMs do not expose precise internal data dependencies. When a model reads a prompt containing both trusted and untrusted text, the system usually cannot determine exactly which tokens influenced the output. FIDES therefore propagates labels conservatively. If a model response is generated from a context containing untrusted data, the response is treated as influenced by untrusted data. This conservative design may restrict some useful actions, but it allows the system to reason safely about tool calls.

4.3 Threat Model and Planner Model

FIDES considers agents that follow the agent-loop paradigm popularized by ReAct-style tool use [6, 32]. The adversary can tamper with data returned by tools or external resources. For example, an attacker may send a malicious email, modify a web page, or provide a document that the agent later reads. The adversary may not directly observe every LLM query, but may observe the effects of some tool calls, such as messages sent to an attacker-controlled server [6].

The planner's design therefore becomes part of the security mechanism. A basic planner maximizes visibility by appending every tool result into the conversation history, but this design also maximizes taint propagation. FIDES studies alternative planner designs that preserve enough information for task completion while limiting how much untrusted data reaches the main planning context. A more careful planner can store tool results in planner memory rather than directly exposing them to the LLM context; the LLM may see a variable reference instead of raw untrusted data. When the planner needs information inside the variable, it can use constrained inspection mechanisms to extract only limited structured information. This design aims to preserve utility while preventing untrusted text from freely influencing the main planning context [6].

4.4 Architecture: Labeled Planning and Policy Enforcement

Figure 4.1 shows the FIDES architecture. The agent loop receives the user's task and coordinates the planner, LLM, tools, planner memory, and policy engine. Data is labeled as it moves through messages, actions, tool calls, arguments, and tool results. When the planner proposes a tool call, the policy engine checks the labels associated with the action and its arguments. A consequential action executes only if it satisfies the relevant security policy [6].

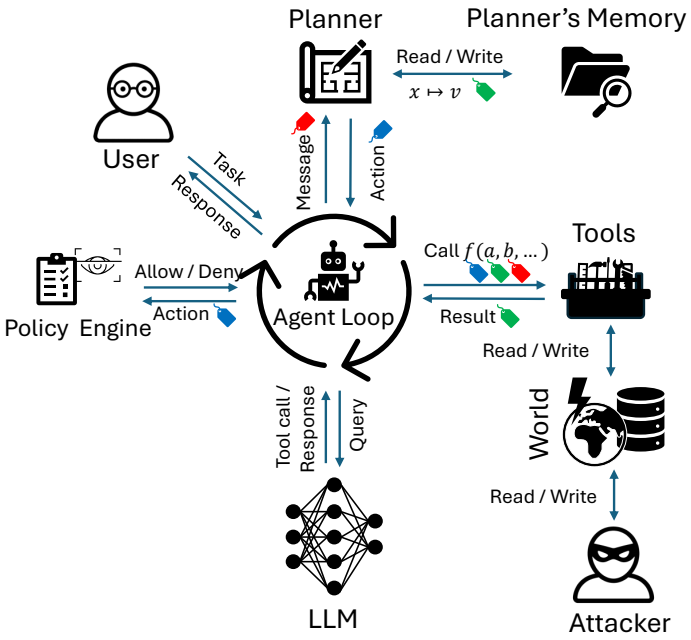

Figure 4.1: Overview of FIDES. The agent loop coordinates the planner, LLM, tools, planner memory, and policy engine. Labels are propagated through messages, actions, tool calls, and results; consequential tool calls execute only if they satisfy policy checks. Adapted from [6].

This architecture has two important differences from a basic ReAct-style loop. First, tool results need not be appended directly into the conversation history. They can be stored in the planner's memory, reducing unnecessary tainting of the main context. Second, the policy engine is external to the LLM. The LLM may propose a tool call, but the system decides whether that call is allowed by evaluating labels and policies deterministically.

FIDES therefore treats the planner as a security-critical component. The planner is not merely an implementation detail that formats prompts and tool calls; it is the part of the system that determines what the model can see, how data labels are propagated, and which action labels are checked. By modifying the planner architecture, FIDES expands the class of tasks that can be completed securely.

4.5 Security Labels and Lattices

FIDES assigns labels to all pieces of data processed by the agent. Labels form a lattice, allowing the system to compute the least upper bound, or join, of multiple labels [6, 8]. When data from two sources is combined, the result receives a label that conservatively reflects both sources. This is essential for safe propagation: if trusted data and untrusted data are combined, the derived value must reflect the presence of untrusted influence.

This propagation rule can be summarized as:

$$\ell(x \cdot y) = \ell(x) \sqcup \ell(y),$$

where $\ell(\cdot)$ denotes the label of a value and $\sqcup$ denotes the lattice join. In other words, data derived from multiple sources inherits the combined confidentiality and integrity restrictions of those sources.

For confidentiality, a simple lattice can distinguish public data from confidential data. More expressive labels can encode sets of authorized readers. If data readable by users

$\{A, B, C\}$ is combined with data readable by users $\{B, C, D\}$, the derived data should be readable only by users allowed to read both sources. For integrity, a simple lattice can distinguish trusted data from untrusted data. More expressive labels can encode possible writers or origins. If data may have been written by an untrusted source, derived data should reflect that low integrity [6].

Figure 4.2 shows the product of the confidentiality and integrity lattices used in FIDES. The bottom represents trusted public data, while the top represents untrusted confidential data. This product lattice allows FIDES to reason about both whether data may have been influenced by an attacker and whether data may be sent to a particular destination.

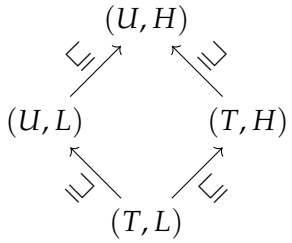

Figure 4.2: Product of confidentiality and integrity lattices in FIDES. Labels allow the planner and policy engine to track whether data is trusted or untrusted and whether it is public or confidential. Adapted from [6].

Labels enter the system through initial messages, tool wrappers, and structured tool results. For example, user instructions may be labeled trusted, external emails may be labeled untrusted, and private documents may be labeled confidential to a set of authorized readers. FIDES therefore depends on meaningful labels and trusted wrappers: if a tool incorrectly labels untrusted content as trusted or confidential data as public, the policy engine may allow unsafe actions [6].

4.6 Two Core Policies: Trusted Action and Permitted Flow

FIDES defines two fundamental policies for agent-tool security: Trusted Action and Permitted Flow. These policies correspond to the two major security concerns in tool-calling agents. Trusted Action protects integrity by preventing attacker-controlled data from triggering consequential actions. Permitted Flow protects confidentiality by preventing private data from being sent to unauthorized recipients [6].

The Trusted Action policy, denoted P-T, permits a consequential tool call only if the model's decision to call the tool is based on trusted data. For example, if a malicious email instructs the agent to delete files or forward private content, the resulting action would be influenced by untrusted data and should be blocked. P-T is therefore designed to prevent indirect prompt injection from triggering harmful actions.

The Permitted Flow policy, denoted P-F, permits data egress only if all recipients are authorized to read the data being sent. For example, if an agent summarizes confidential email content, P-F checks whether the destination recipient is allowed to receive that content. This addresses data exfiltration even when the model is confused or manipulated.

Table 4.1: FIDES core policies.

Policy	Security Goal	What It Blocks
P-T	Ensure consequential actions are based only on trusted sources.	Prompt-injected or untrusted content triggering actions such as deletion, sending, or modification.
P-F	Ensure egressed data goes only to authorized recipients.	Confidential data being sent to recipients not authorized to read it.
P-T + P-F	Protect both action integrity and data confidentiality.	Attacks that both trigger unauthorized actions and exfiltrate sensitive data.

Table 4.1 summarizes these policies. The distinction between them is important because not all tools have the same security role. A tool that modifies state, such as deleting a file, primarily needs an integrity check. A tool that sends data outside the system, such as sending an email or making a web request, needs a confidentiality check. Some tools require both.

4.7 Selective Hiding and Constrained Inspection

A naive information-flow design would hide all untrusted data from the planner or block all actions after untrusted data appears. Such a design would be secure but often useless. Many legitimate tasks require inspecting untrusted content. For example, a user may ask the agent to summarize external emails, process a customer support message, or extract tasks from a document. If the planner cannot inspect untrusted data at all, it cannot complete these tasks.

FIDES addresses this problem through selective hiding and constrained inspection. Selective hiding stores tool results in planner memory when exposing them would raise the security label of the planner's context and restrict later tool calls. Instead of placing raw untrusted content directly into the conversation history, the planner stores it in a variable and exposes only a reference to the LLM. This reduces unnecessary tainting of the main context [6].

Constrained inspection allows the planner to extract limited structured information from hidden variables. FIDES can use a quarantined LLM to inspect a variable and return a result of a constrained type, such as a Boolean, enum, or structured field. For example, instead of showing the full malicious email to the main planner, the system may ask a quarantined model whether the email is urgent and receive only a Boolean answer. A single Boolean value has far less capacity to carry an injected instruction than the full text of the email.

This design is related to the Dual LLM pattern, which separates a privileged model from a quarantined model that handles untrusted content [28]. FIDES incorporates a similar intuition but embeds it within a label-tracking and policy-enforcement architecture. The quarantined inspection result is still labeled and constrained, so the system can reason about how it may influence later actions.

4.8 Evaluation on AgentDojo

FIDES evaluates its planner designs on AgentDojo, a benchmark for tool-calling agents under prompt-injection attacks [6, 7]. The evaluation compares planner variants with and without policy checks and studies how selective hiding and revealing affect the security-utility trade-off. The key question is whether FIDES can stop practical prompt-injection attacks while still completing useful tasks.

The reported findings support the central design. With policy checks enabled, FIDES stops all prompt-injection attacks in AgentDojo. Without policy checks, all planners, including FIDES, succumb to practical prompt-injection attacks. This shows that the planner architecture alone is not enough; deterministic policy enforcement is essential [6].

Table 4.2: FIDES evaluation summary on AgentDojo.

Finding	Reported Result
Attack blocking	With policy checks enabled, FIDES stops all prompt-injection attacks in AgentDojo.
No policy checks	Without policy checks, all planners, including FIDES, succumb to practical prompt-injection attacks.
Reasoning-model utility	FIDES completes about 16% more tasks than a basic planner.
Prompt-tuned utility	The improvement rises to about 24%, approaching the human-oracle setting.
Hiding/revealing overhead	With policy checks disabled, selective hiding and revealing do not reduce overall task completion for reasoning models.

Table 4.2 summarizes the main reported findings. The key lesson is that planner architecture and policy enforcement are both necessary: planner mechanisms improve expressiveness, but the deterministic policy checks are what stop attacks [6].

These results indicate that information-flow control need not simply block useful behavior. A carefully designed planner can avoid unnecessary tainting, inspect hidden information in constrained ways, and enforce security policies at the tool-call boundary. The result is a more expressive secure planner than a design that either exposes all tool outputs to the LLM or hides all untrusted data from it.

4.9 Strengths

FIDES has several strengths.

First, it gives a formal security interpretation to agent-tool interaction. Rather than treating prompt injection as only an input-filtering problem, FIDES models how messages, actions, tool calls, results, and planner memory carry labels. This makes it possible to state security policies in terms of integrity and confidentiality rather than relying only on natural-language safety judgments.

Second, FIDES separates planning from enforcement. The LLM may propose an action, but the policy engine checks whether the action satisfies label-based constraints. This is similar in spirit to operating-system security mechanisms that mediate access to resources even when applications are buggy or compromised [21].

Third, FIDES improves expressiveness through planner design. Selective hiding and constrained inspection allow the agent to use untrusted information without fully tainting the main planning context. This is important because many useful tasks require processing untrusted data. A secure agent cannot simply ignore all external content.

Fourth, FIDES addresses both integrity and confidentiality. Trusted Action prevents untrusted data from triggering consequential actions, while Permitted Flow prevents confidential data from being sent to unauthorized recipients. This dual focus makes FIDES broader than defenses that address only prompt-injection success rates.

4.10 Limitations

FIDES also has important limitations. These limitations identify the boundary of what information-flow control can guarantee in agentic systems.

The first limitation is label correctness. FIDES assumes that tool wrappers and system components attach meaningful labels to data. If a tool incorrectly labels untrusted content as trusted or confidential data as public, the policy engine may allow unsafe actions. In practice, building and maintaining correct labels across many services and APIs may be difficult.

The second limitation is conservativeness. Because LLMs are opaque, FIDES cannot precisely determine which parts of a prompt influenced a model output. It therefore propagates labels conservatively. This is safe, but it can overtaint the planner context and block useful actions. Selective hiding mitigates this problem, but it does not remove the underlying tension between security and utility.

The third limitation is planner complexity. FIDES requires a planner architecture with labeled messages, planner memory, tool wrappers, policy checks, and constrained inspection. This is more complex than a basic agent loop. Deploying such a system in real-world agent platforms would require careful engineering and integration with existing tool ecosystems.

The fourth limitation is policy scope. FIDES can prevent untrusted influence and illicit data flows according to its labels and policies, but it does not decide whether an action is contextually appropriate for the user's current purpose. Nor does it compute the least set of service permissions required for a task. These are complementary problems handled by Conseca and MiniScope in this report's framework.

4.11 Position in the System-Level Security Stack

Within the framework of this report, FIDES provides information-flow control. It asks whether the information influencing an action is trusted and whether confidential data flows only to authorized recipients. This is a different question from Conseca's purpose control. An action may be appropriate for the user's task but still unsafe if it was triggered by attacker-controlled content or sends confidential data to the wrong place. Conversely, a data flow may be permitted by labels but still be inappropriate for the user's current purpose.

FIDES therefore occupies the secure-by-design and information-flow layer of the system-level defense landscape. It changes the structure of the agent planner so that data is labeled, hidden when necessary, inspected in constrained ways, and checked by a policy engine before tool execution. In a layered secure agent runtime, FIDES-like enforcement could sit below contextual action policies, ensuring that the data used to justify an action and the data sent through tools satisfy integrity and confidentiality constraints.

4.12 Summary

This chapter analyzed FIDES as a representative information-flow-control system for secure agent-tool interaction. FIDES begins from the observation that ordinary agent loops mix trusted and untrusted information inside a shared conversation history, allowing attacker-controlled content to influence later tool calls. To address this, FIDES labels data with confidentiality and integrity metadata, propagates labels through the planner and tool calls, and deterministically enforces Trusted Action and Permitted Flow policies.

The main contribution of FIDES is to bring classical information-flow-control reasoning into the design of LLM agent planners. Its mechanisms show that secure agents require

not only better prompts, but also planner architectures that track provenance, hide risky information, inspect it safely, and mediate consequential tool calls. However, FIDES does not solve the problem of overprivileged service access. Chapter 5 turns to MiniScope, which addresses that problem through least-privilege authorization.

Chapter 5

Authority Control with MiniScope

5.1 Motivation: Least Privilege for Tool-Calling Agents

Chapter 4 studied information-flow control: whether untrusted or confidential data can improperly influence actions or escape to unauthorized recipients. This chapter studies the third control plane in secure agent-tool interaction: authority control. Authority control asks whether the agent has only the minimum permissions needed to complete the user's task.

This problem arises because tool-calling agents increasingly operate over real user services. Modern assistants can connect to email, calendar, cloud storage, messaging, note-taking, and productivity platforms. Once connected, an agent may receive service credentials or OAuth tokens that allow it to call APIs on the user's behalf. If those permissions are broader than the task requires, then any model mistake, prompt injection, or malicious tool response can cause greater damage. MiniScope begins from this problem: an unreliable LLM-based agent should not be trusted with broad user credentials when a narrower set of permissions is sufficient [34].

The principle of least privilege states that a program should operate with only the privileges necessary to complete its task [21]. This principle is standard in operating systems and access-control design, but tool-calling agents make it difficult to apply. Agentic tasks are expressed in natural language, may require multiple tool calls, and may span several services. A user request such as "check my email and add relevant meetings to my calendar" may involve reading email, identifying calendar-related content, and creating calendar events. The system must infer which permissions are needed without granting the agent full access to all connected services.

MiniScope addresses this problem by acting as a firewall between an untrusted agent and the user's external services. Instead of giving the agent direct credentials, MiniScope stores credentials separately, tracks granted permissions, infers the minimal permission set needed for the agent's execution plan, and mechanically checks each tool call before forwarding it to the service [34]. Its main contribution is to make least privilege enforceable for dynamic tool-calling tasks.

5.2 Threat Model and User-Agent-Service Model

MiniScope focuses on the user-agent-service model. A user interacts with an AI agent, and the agent uses external services to complete tasks. The services may include applications such as Gmail, Google Calendar, Google Drive, Slack, Dropbox, Outlook, Notion, or Zoom. The agent receives a natural-language request, determines which tools to call, invokes those

tools, processes the responses, and returns a result to the user [34].

The threat model treats the LLM agent as untrusted. The model may hallucinate, misinterpret user intent, or be influenced by prompt injection. Tool responses from services are also treated as untrusted because they may contain attacker-controlled content. The user is trusted as the ground-truth authority for granting or denying permissions. Services are assumed to implement their API methods according to their specifications, and MiniScope enforces security at the granularity of the permission hierarchy [34].

Figure 5.1 shows the MiniScope architecture. MiniScope sits between the agent and external services. It isolates user credentials from the agent, prompts the user when additional permissions are needed, and forwards tool calls to services only if they are authorized under the granted permissions. In this architecture, the agent may propose tool calls, but it does not directly hold the credentials needed to execute them.

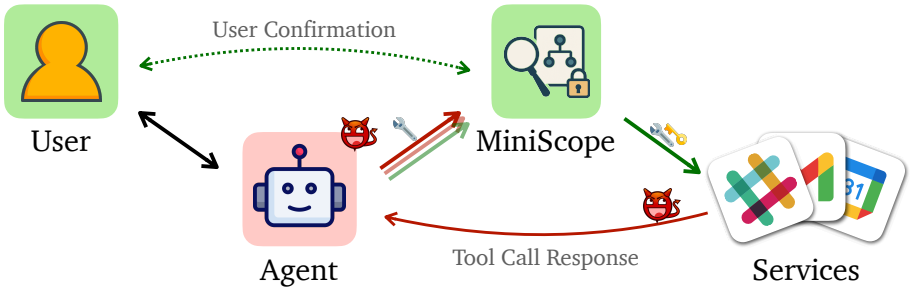

Figure 5.1: MiniScope architecture. MiniScope mediates between an untrusted agent and external services, isolates user credentials, prompts the user for additional permissions when needed, and forwards only authorized tool calls. Adapted from [34].

This design shifts the security boundary from the LLM to the authorization layer. Even if the agent proposes an incorrect or malicious tool call, MiniScope checks whether the call is authorized under the user's current permission state. If not, the call is rejected or the user is asked to approve additional permissions. The goal is not to make the LLM fully trustworthy, but to confine the damage it can cause through service APIs.

5.3 Why Existing Permission Models Are Insufficient

Existing agent deployments often rely on coarse permission patterns. One pattern is per-call confirmation: the user must approve every tool invocation. This provides fine-grained control but interrupts the workflow and creates confirmation fatigue. Another pattern is high-risk confirmation: only actions judged risky require approval. This reduces interruptions but depends on reliable risk classification. A third pattern is initial confirmation, where the user grants a fixed set of connector permissions when connecting the service. This improves usability but often leads to overprivilege. A final pattern is manual configuration, where the user or developer configures permissions for custom tools; this is flexible but error-prone [34].

These patterns expose a security-usability trade-off. Asking the user too often reduces usability and may cause careless approvals. Asking the user too rarely improves usability but may grant the agent excessive authority. MiniScope's goal is to reduce this trade-off by using a structured permission hierarchy and automated least-privilege reasoning.

This design is inspired partly by mobile operating-system permission models, where applications request access to resources such as location, camera, contacts, or storage, and the user grants or denies those permissions [1, 34]. However, agentic systems differ from mobile apps because the exact tool calls needed for a natural-language task may not be

known in advance. MiniScope therefore needs a task-level mechanism for inferring required permissions from an execution plan.

5.4 Permission Hierarchies from OAuth Scopes

The central design idea in MiniScope is to reconstruct permission hierarchies over tool calls. This hierarchy is related to hierarchical role-based access control, where broader roles inherit the permissions of narrower roles [22]. Many real-world services already define OAuth scopes that group API methods by functionality and sensitivity.

MiniScope uses these existing scopes to derive a hierarchy. If one permission group supports more tool calls than another, it corresponds to broader authority and is therefore considered more sensitive. A narrower scope should be preferred when it is sufficient for the task. This turns least privilege into a structured optimization problem: given a set of tool calls in an execution plan, choose the least sensitive set of permission scopes that authorizes those calls [34].

Figure 5.2 shows an example permission hierarchy reconstructed for Google Calendar. The figure illustrates why naming alone is not enough. A scope's textual name may suggest one hierarchy, but the actual hierarchy depends on which API methods the scope authorizes. MiniScope therefore derives the hierarchy from capability relationships rather than relying only on developer intuition or natural-language scope names [34].

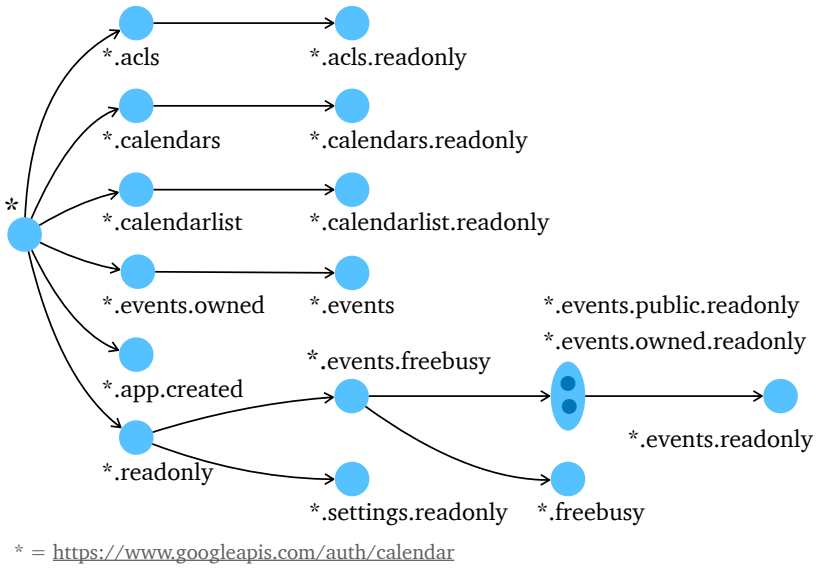

Figure 5.2: Google Calendar permission hierarchy reconstructed by MiniScope. The hierarchy is derived from the API methods authorized by each OAuth scope rather than only from scope names. Adapted from [34].

This hierarchy provides two benefits. First, it gives MiniScope a structured way to reason about permission sensitivity. Second, it allows the system to compare alternative permission sets. If multiple scopes can authorize the same method, MiniScope can prefer the combination that minimizes excess authority.

5.5 ILP-Based Least-Privilege Solver

Once MiniScope has a permission hierarchy, it must determine which scopes are required for a task. A simple heuristic might choose the most restrictive scope for each tool call

independently and then take the union. However, MiniScope shows that this approach is not always optimal because a single method can often be authorized through multiple scope paths. In Google Calendar, for example, many API methods can be authorized through more than one scope. Local greedy choices may therefore fail to minimize the overall permission set [34].

MiniScope formulates the least-privilege mapping problem as an integer linear programming problem. Given a permission tree or forest and an execution plan consisting of API calls, the solver selects permission nodes that cover the required calls while minimizing a cost function. The cost of selecting a permission can reflect the breadth or sensitivity of the scope, such as the number of API methods it authorizes. Previously granted permissions can be fixed as already selected so that the solver preserves the user's existing permission state [34].

Let $T = (V, E)$ denote the permission hierarchy, where each node $v \in V$ represents a permission scope. Let the execution plan require API calls $e_1, \ldots, e_n$, and let S_i be the set of scopes that can authorize call e_i. MiniScope introduces a binary decision variable x_v for each scope:

$$x_v = \begin{cases} 1, & \text{if permission scope } v \text{ is selected,} \\ 0, & \text{otherwise.} \end{cases}$$

The least-privilege selection problem can then be written as:

$$\min \sum_{v \in V} \text{cost}(v) x_v$$

subject to:

$$\forall S_i, \quad \sum_{v:\text{subtree}(v) \cap S_i \neq \emptyset} x_v \geq 1.$$

The objective minimizes the cost of selected scopes, while the constraint ensures that every required API call is covered by at least one selected permission. This formalization is useful because it handles cases where an API method can be authorized through multiple scope paths, where simple greedy selection may fail.

This formulation is important because it makes least privilege mechanical rather than advisory. Instead of prompting an LLM to "choose minimal permissions" or asking the user to reason about many API scopes, MiniScope uses the hierarchy and solver to compute a permission set. The LLM may propose an execution plan, but it does not decide which credentials it receives. That decision is made by a deterministic authorization layer.

Table 5.1: MiniScope's system design at a glance.

Aspect	MiniScope Design
Security goal	Ensure that a tool-calling agent receives only the minimum service permissions needed for a task.
Main mechanism	Reconstruct permission hierarchies from OAuth scopes and solve for least-privilege permissions.
Protected boundary	Boundary between the untrusted agent and external user services.
Trusted inputs	User permission decisions, MiniScope enforcement logic, and service authorization specifications.
Untrusted inputs	LLM agent behavior, tool-call plans, service responses, and prompt-injected content.
Guarantee style	Executed tool calls are permitted under the user's current granted permission state.
Primary limitation	Protection is limited by the granularity and correctness of the permission hierarchy.

Table 5.1 summarizes MiniScope's design. The key distinction is that MiniScope does not attempt to make the LLM reliable. It assumes the agent may be unreliable and confines its authority at the service boundary.

5.6 End-to-End Enforcement Workflow

MiniScope's runtime workflow has four main steps. First, the user submits a natural-language task to the agent. Second, the agent proposes an execution plan containing one or more tool calls. Third, MiniScope determines which permissions are needed for that plan by consulting the permission hierarchy and least-privilege solver. If the current permission state is insufficient, MiniScope prompts the user to grant the additional minimal permissions. Fourth, each tool call issued by the agent is mechanically checked before it is forwarded to the corresponding service [34].

This workflow combines automation with user authority. The user is not asked to approve every tool call, but also does not grant broad permissions blindly at the beginning of the interaction. Instead, the system prompts the user when the task requires additional authority. Once the user grants a permission, MiniScope records it and uses it for later checks. This resembles mobile-style permission management, but adapted to dynamic agentic workflows.

Credential isolation is central to this workflow. The agent can request a tool call, but it cannot independently use the user's tokens. MiniScope forwards the request only if the permission check succeeds, reducing the risk that prompt injection or model mistakes directly expose credentials or enable unauthorized service calls [34].

5.7 Evaluation

MiniScope evaluates permission minimality, runtime overhead, and user effort. The evaluation uses a synthetic dataset derived from ten real-world applications: Gmail, Google Calendar, Google Drive, Google Storage, Slack, Dropbox, Outlook, Outlook Calendar, Notion, and Zoom. The dataset includes tasks involving a single tool call, multiple tool calls within one application, and multi-application workflows. This design is intended to reflect realistic tool-calling complexity more closely than simplified toy benchmarks [34].

The first evaluation dimension is permission minimality. MiniScope compares its least-privilege solver against an LLM-based baseline that attempts to decide minimal permissions. The result supports MiniScope's argument that least privilege should be enforced mechanically rather than delegated to an LLM [34].

The second and third dimensions are runtime overhead and user effort. MiniScope reports low overhead relative to a vanilla tool-calling agent and uses simulated Gmail interactions to estimate how often users must confirm additional permissions. Table 5.2 summarizes these findings.

Table 5.2: MiniScope evaluation summary.

Evaluation Dimension	Reported Result
Permission minimality	LLM-based baselines reach 70–83% optimality with proprietary models and 20–34% with open-source models.
Runtime overhead	MiniScope adds only 1–7% overhead relative to a vanilla tool-calling agent.
Cost comparison	The LLM-in-the-loop baseline incurs orders-of-magnitude higher overhead and additional estimated cost per request.
User confirmation	Simulated Gmail interactions produce confirmation rates between 18% and 60%, depending on user persona.
Connector audit	MiniScope identifies six overprivileged deployments in real-world connectors.

Table 5.2 summarizes the main reported findings. The key lesson is that least-privilege reasoning can be both mechanical and practical. MiniScope improves permission minimality without imposing large runtime overhead, while its mobile-style confirmation model reduces user burden compared with approving every tool call.

5.8 Strengths

MiniScope has several strengths.

First, it treats agent authorization as a systems problem rather than a prompt-engineering problem. The LLM does not decide what permissions it should receive. Instead, MiniScope uses authorization specifications, permission hierarchies, and a solver to determine which scopes are needed.

Second, MiniScope provides a practical way to apply least privilege to real services. By deriving permission groups from OAuth scopes, it leverages structures already present in many service ecosystems. This makes the approach more deployable than a system that requires every application to design a new permission model from scratch.

Third, MiniScope separates credentials from the agent. The agent may propose tool calls, but it does not directly hold the user's credentials. This reduces the risk that prompt injection or model mistakes directly expose access tokens or enable unauthorized service calls.

Fourth, MiniScope addresses usability. Pure per-call confirmation can overwhelm users, while one-time broad consent can overprivilege agents. MiniScope's hierarchical permission model offers a middle ground: users grant additional authority only when needed, and the system tries to make that additional authority minimal.

5.9 Limitations

MiniScope also has important limitations. These limitations identify the boundary of what authority control can guarantee.

The first limitation is permission granularity. MiniScope can enforce least privilege only at the granularity provided by the permission hierarchy. If an OAuth scope is broad, then the least available permission may still authorize more actions than the user actually intended. MiniScope can reduce overprivilege, but it cannot create fine-grained distinctions that the underlying service does not expose.

The second limitation is plan dependence. MiniScope infers required permissions from the agent's proposed execution plan. If the plan is incomplete or changes during execution,

additional permissions may be needed later. This is manageable through iterative confirmation, but it means permission inference is tied to the quality and stability of the plan.

The third limitation is that MiniScope does not prevent prompt injection in general. A prompt-injected agent may still issue harmful tool calls that fall within already granted permissions [13, 33, 34]. MiniScope confines the damage to the granted scope, but it does not determine whether the action is contextually appropriate or whether it was influenced by untrusted data. Those problems correspond to Conseca and FIDES in this report's framework.

The fourth limitation is service dependency. MiniScope assumes service authorization specifications are available and that service methods behave according to their specifications. Incorrect specifications, inconsistent OAuth scopes, or poorly documented APIs can reduce the quality of the reconstructed hierarchy.

5.10 Position in the System-Level Security Stack

Within the framework of this report, MiniScope provides authority control. It asks whether the agent has only the minimal delegated authority required for the task. This is different from Conseca's purpose control and FIDES's information-flow control. An action may be contextually appropriate and based on trusted data, but the agent should still not hold broad service permissions if a narrower scope is sufficient. Conversely, even a least-privileged permission set cannot determine whether a particular allowed action is appropriate for the current task or safe with respect to information flow.

MiniScope therefore occupies the identity-and-access-management layer of the system-level defense landscape. It mediates the relationship between an unreliable agent and real user services, isolates credentials, and enforces permission checks before tool calls are executed. In a layered secure agent runtime, MiniScope-like enforcement could sit at the service boundary, limiting the authority available to the planner and executor regardless of what the LLM proposes.

5.11 Summary

This chapter analyzed MiniScope as a representative authority-control system for secure agent-tool interaction. MiniScope begins from the observation that tool-calling agents often operate over sensitive user services and may receive broader permissions than a task requires. To address this, MiniScope reconstructs permission hierarchies from OAuth scopes, uses an ILP-based solver to infer least-privilege permissions, isolates credentials from the agent, and mechanically checks tool calls before forwarding them to services.

The main contribution of MiniScope is to make least privilege operational for dynamic agentic tasks. It does not try to solve all prompt-injection or contextual-action problems. Instead, it confines the agent's authority so that mistakes or attacks have a smaller blast radius. Together with Conseca and FIDES, MiniScope completes the three-control-plane view developed in this report: purpose control, information-flow control, and authority control.

Chapter 6

Cross-Paper Synthesis

6.1 Revisiting the Three Control Planes

The previous three chapters analyzed Conseca, FIDES, and MiniScope as representative system-level defenses for tool-calling agents. Although all three papers address the same broad problem of securing agent-tool interaction, they do not solve the same security problem. Conseca focuses on whether a proposed action is appropriate for the user's current task. FIDES focuses on whether the information influencing an action is trusted and whether confidential data flows only to authorized recipients. MiniScope focuses on whether the agent has only the minimal service authority required for a task [6, 25, 34].

This distinction is central to the argument of this report. Agent-tool security cannot be reduced to a single enforcement mechanism because the agent-tool boundary contains several different failure modes. A tool call may be inappropriate even if it is authorized. A tool call may be authorized and contextually appropriate, but still unsafe if it was triggered by attacker-controlled content. A tool call may be safe with respect to information flow, but still reveal a deployment problem if the agent holds broader credentials than the task requires. These cases correspond to three different control planes: purpose, information flow, and authority.

Table 6.1: Comparison of the three focal systems.

Dimension	Conseca	FIDES	MiniScope
Control plane	Purpose control	Information-flow control	Authority control
Protected focus	Action appropriateness	Data influence and data egress	Delegated service permissions
Main question	Is this action justified by the current task?	Did unsafe data influence the action or flow to the wrong recipient?	Does the agent have only the permissions required for the task?
Mechanism	Contextual policy generation and deterministic enforcement	Confidentiality and integrity labels, planner memory, and policy checks	OAuth-derived permission hierarchy and ILP-based least-privilege solver
Assumptions	Trusted context and isolated policy generator	Correct labels, trusted wrappers, and policy engine	User decisions, service specifications, and MiniScope enforcement
Guarantee	Actions stay within the generated contextual policy	Consequential actions and egress obey label-based policies	Executed tool calls are authorized under granted permissions
Main limitation	Generated policy may be incomplete or based on limited trusted context	Labels may be coarse or incorrect; conservative tainting can reduce utility	Permission scopes may be coarse; attacks within granted scope remain possible

Table 6.1 summarizes the comparison. The key point is that each system places deterministic enforcement around a different aspect of the agent runtime: purpose, information flow, or authority.

6.2 From Component Hardening to System-Level Enforcement

A common theme across the three papers is the move away from relying solely on the LLM's internal behavior. Component-level defenses such as alignment, prompt filtering, input-output guardrails, and tool vetting can reduce the probability of unsafe behavior, but they do not by themselves create a complete security boundary. The focal papers instead place enforcement mechanisms around the model: Conseca uses a deterministic policy enforcer, FIDES uses a label-based policy engine, and MiniScope uses a mechanical permission checker [6, 25, 34].

This shift resembles a familiar lesson from operating systems. A secure operating system does not assume that every application is correct or benign. Instead, it mediates access to files, memory, devices, credentials, and networks. Similarly, a secure agent runtime should not assume that the LLM planner will always interpret instructions correctly or resist adversarial manipulation. It should mediate access to tools, private data, external APIs, and delegated credentials. This is why system-level enforcement is a natural direction for agent security. This also aligns with the observation that the dangerous combination for agents is the simultaneous presence of private data, untrusted content, and external communication channels [29].

The three systems differ in how they define the object of enforcement. In Conseca, the object is the proposed action and its arguments relative to the current task. In FIDES, the object is the labeled flow of information through planner state and tool calls. In MiniScope, the object is the authorization relationship between tool calls and service permissions. These

differences matter because each system can block attacks that the others may not catch.

For example, suppose an agent receives a malicious email instructing it to forward private files to an attacker. Conseca may block the action if forwarding those files is outside the task-specific contextual policy. FIDES may block the action because the decision was influenced by untrusted email content or because confidential data would flow to an unauthorized recipient. MiniScope may block the action if the agent lacks the necessary permission scope for file reading or email sending. The same attack can therefore be constrained at multiple layers, each using a different security principle.

6.3 Complementarity and Layering

The strongest interpretation of the three papers is not that one replaces the others, but that they suggest a layered agent security stack. In such a stack, MiniScope-like authority control limits what the agent can access at the service boundary. FIDES-like information-flow control tracks whether data from trusted and untrusted sources can influence consequential actions or egress. Conseca-like purpose control checks whether the remaining proposed action is appropriate for the user's current task.

The order of these layers is not fixed, but their functions are distinct. Authority control reduces the blast radius of mistakes by limiting the permissions available to the agent. Information-flow control prevents untrusted or confidential information from crossing dangerous boundaries. Purpose control ensures that even technically permitted actions remain aligned with the user's current task. A secure runtime may need all three because each layer answers a different question.

Consider a task where a user asks an agent to summarize work emails and send a brief update to a manager. MiniScope can ensure that the agent receives only email-read and email-send permissions necessary for the task, rather than full mailbox or account authority. FIDES can ensure that emails from untrusted senders do not trigger consequential actions and that confidential content is not sent to unauthorized recipients. Conseca can ensure that the agent sends only the type of update justified by the task and does not perform unrelated actions such as deleting messages or forwarding attachments. The combined system would provide a stronger security boundary than any single layer alone.

This layered view also clarifies the limitations of each system. Conseca can decide that an action does not fit the current purpose, but it does not fully track data provenance or compute least-privilege credentials. FIDES can enforce integrity and confidentiality policies over labeled data, but it does not determine whether a task-specific action is socially or contextually appropriate. MiniScope can restrict service authority, but it does not stop harmful behavior that remains inside already granted permissions. The systems are therefore best understood as complementary partial solutions.

6.4 Security Guarantees and Their Boundaries

The three papers all aim for more rigorous guarantees than prompt-only defenses, but their guarantees have different boundaries. Conseca guarantees that no external tool action executes outside the generated contextual policy, assuming the policy generator, trusted-context boundary, and enforcer integration are correct [25]. This is a runtime safety property over proposed actions, but it is only as good as the generated policy.

FIDES provides label-based integrity and confidentiality guarantees. Its Trusted Action policy prevents consequential actions from being triggered by untrusted data, while its Permitted Flow policy prevents confidential data from being sent to unauthorized recipients [6]. These guarantees are stronger than natural-language safety judgments because they are

enforced by a policy engine, but they depend on correct labels, trusted tool wrappers, and conservative propagation through the planner.

MiniScope guarantees that executed tool calls are permitted under the user's current granted permission state [34]. This guarantee is mechanical and does not depend on the LLM correctly interpreting least privilege. However, it operates at the granularity of available permission scopes. If a scope is broad, then the least available permission may still allow more behavior than the user intended.

Thus, each guarantee has a boundary:

- Conseca's boundary is the generated policy.
- FIDES's boundary is the label model and policy engine.
- MiniScope's boundary is the permission hierarchy and granted scope state.

This observation is important because system-level security does not eliminate assumptions. It makes assumptions explicit and moves enforcement into mechanisms that can be inspected, audited, and tested. Compared with an LLM-only defense, this is a significant improvement because the security boundary is no longer hidden inside model behavior.

6.5 Security-Utility Trade-offs

All three systems face a security-utility trade-off. The difference lies in where the trade-off appears.

In Conseca, the trade-off is between policy strictness and task completion. A restrictive policy can block dangerous actions but may prevent the agent from completing legitimate tasks. A permissive policy can preserve utility but allow harmful behavior. Conseca attempts to improve this trade-off by generating contextual policies that are specific to the user's current task [25].

In FIDES, the trade-off is between conservative taint propagation and planner utility. If all untrusted data is exposed to the planner, the agent can reason over rich context but becomes vulnerable to prompt injection and unsafe influence. If all untrusted data is hidden or treated as fully tainted, the agent may become safe but unable to complete tasks involving external information. FIDES addresses this trade-off through selective hiding and constrained inspection [6].

In MiniScope, the trade-off is between permission granularity and user burden. Fine-grained per-call confirmation can reduce overprivilege but interrupts the user. Broad one-time authorization improves usability but increases the agent's blast radius. MiniScope uses permission hierarchies and least-privilege solving to reduce unnecessary permissions while avoiding per-call confirmation for every action [34].

These trade-offs are not incidental implementation issues. They are fundamental to the design of secure agentic systems. Agents are valuable precisely because they can act flexibly in open-ended contexts. Security mechanisms must therefore constrain behavior without reducing the agent to a static script. The three papers explore different points in this design space.

6.6 Evaluation Methodology Across the Papers

The evaluation styles of the three papers also differ. Conseca uses a proof-of-concept Linux computer-use agent with filesystem and email tools. Its evaluation emphasizes whether contextual policies can preserve task completion while denying inappropriate actions [25].

This is appropriate for a HotOS-style position and prototype paper, but the evaluation is small-scale.

FIDES uses AgentDojo to evaluate prompt-injection security and task completion under different planner designs [6]. Its evaluation is broader than Conseca's because it uses a benchmark designed for tool-calling agents under attack. It also compares planner variants with and without policy checks, which helps isolate the role of deterministic enforcement.

MiniScope evaluates permission minimality, runtime overhead, and user effort using a synthetic dataset derived from ten real-world applications [34]. Its evaluation is more deployment-oriented because the system targets real service permissions and OAuth-like authorization structures. Instead of focusing mainly on attack blocking, it measures whether least-privilege permissions can be inferred accurately and efficiently.

These evaluations are difficult to compare directly because they measure different outcomes. Conseca measures contextual appropriateness and task completion. FIDES measures attack blocking and secure task completion. MiniScope measures permission minimality, overhead, and confirmation burden. A direct numerical comparison would therefore be misleading. A better comparison is qualitative: each paper evaluates the property that corresponds to its control plane.

This also reveals an open benchmarking problem. Future agent-security benchmarks should evaluate multiple dimensions together: task utility, attack resistance, information-flow safety, permission minimality, runtime overhead, and user burden. Current benchmarks tend to emphasize only part of this space. As a result, a system may appear strong under one metric while leaving other control planes underprotected.

6.7 Toward a Layered Agent Runtime

The synthesis above suggests a concrete runtime architecture. At the service boundary, an authority layer would isolate credentials and enforce least-privilege permissions. Inside the agent loop, an information-flow layer would label tool results, planner memory, and outgoing actions. Between the planner and executor, a contextual policy layer would check whether proposed actions are appropriate for the current user purpose.

Such a runtime would not require trusting the LLM as the sole security authority. The LLM could still plan, summarize, reason, and propose actions, but external mechanisms would constrain what it can see, what it can influence, what it can send, and what authority it can exercise. This separation preserves the usefulness of LLM-based planning while restoring some of the mediation principles that traditional systems security relies on.

A layered runtime would also make failures easier to diagnose. If an action is blocked by MiniScope, the issue is insufficient authority. If it is blocked by FIDES, the issue is unsafe information flow or untrusted influence. If it is blocked by Conseca, the issue is contextual mismatch with the task. These different denial reasons could be logged, audited, and potentially explained to users or developers.

However, layering also introduces new design challenges. Policies may conflict. A contextual policy may allow an action that information-flow labels block. A permission layer may grant a scope that is broader than the contextual policy would ideally allow. A system must decide how to compose these layers, how to present denials to the user, and how to avoid excessive friction. This is why future work should study not only individual defenses, but also their composition.

6.8 Open Challenges

The comparison suggests several open challenges for system-level agent security.

Table 6.2: Open challenges suggested by the three focal systems.

Challenge	Why It Matters	Related Systems
Policy composition	Layered defenses may produce conflicting or redundant decisions.	Conseca, FIDES, MiniScope
Trusted context	Contextual policies require enough trusted information to be useful but not enough to be attacker-controlled.	Conseca
Label correctness	Information-flow guarantees depend on meaningful labels and trusted tool wrappers.	FIDES
Permission granularity	Least privilege is limited by the scopes exposed by services.	MiniScope
User burden	Confirmations, denials, and policy explanations must not overwhelm users.	Conseca, MiniScope
Benchmarking	Security, utility, permission minimality, and user effort should be evaluated together.	Conseca, FIDES, MiniScope
Multi-agent delegation	Future agents may delegate tasks across agents with different users, tools, and permissions.	FIDES, MiniScope
Dynamic adaptation	Policies, labels, and permissions may need to evolve as tasks unfold.	Conseca, FIDES, MiniScope

Table 6.2 summarizes these challenges. The most important is policy composition. Each focal system defines a local enforcement logic, but a real deployment may need all three. If multiple layers disagree, the runtime needs a principled conflict-resolution strategy. Conservative composition may be safe but could block too many tasks. Permissive composition may preserve utility but weaken the security guarantees.

Another challenge is granularity. Conseca depends on the granularity of trusted context and policy constraints. FIDES depends on the granularity of labels. MiniScope depends on the granularity of service permission scopes. Coarse granularity often improves usability and deployability, but weakens security. Fine granularity improves security but increases engineering complexity and user burden.

A third challenge is evaluation. Agent-security systems need benchmarks that capture realistic workflows, adversarial manipulation, user effort, and system-level side effects. A benchmark that measures only prompt-injection success may miss permission overbreadth. A benchmark that measures only permission minimality may miss unsafe information flow. A benchmark that measures only task completion may reward unsafe permissiveness. Future evaluation should therefore be multidimensional.

6.9 Summary

This chapter synthesized Conseca, FIDES, and MiniScope under the three-control-plane framework developed in this report. The main conclusion is that these systems are complementary rather than competing: Conseca constrains purpose, FIDES constrains information flow, and MiniScope constrains delegated authority. Together, they suggest a future direction for secure agent runtimes: layered system-level enforcement around an unreliable but useful LLM planner.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

Tool-calling agents mark an important shift in the deployment of large language models. A standalone LLM mainly produces text, but an agent can read private data, call external services, modify state, and act on behalf of a user. This shift makes security failures more consequential. Prompt injection, hallucination, or overbroad authorization can now lead not only to incorrect text, but also to data leakage, unauthorized actions, and real changes in user systems.

This report argued that securing such agents requires a move from component-level hardening toward system-level enforcement. Model alignment, prompt filtering, tool vetting, and human confirmation remain useful, but they do not by themselves provide a complete security boundary. The central security question is not only whether the LLM is likely to behave safely, but whether the surrounding runtime can constrain what the agent sees, what it can influence, what it can send, and what authority it can exercise.

Through a comparative review of Conseca, FIDES, and MiniScope, this report organized agent-tool security into three control planes: purpose control, information-flow control, and authority control [6, 25, 34]. Conseca represents purpose control by generating contextual policies and enforcing them at runtime. FIDES represents information-flow control by tracking confidentiality and integrity labels through the planner and tool calls. MiniScope represents authority control by computing least-privilege permissions and mediating access to user services.

The main conclusion is that these systems are complementary rather than competing. Conseca answers whether an action fits the user's current task. FIDES answers whether the action or data flow is safe with respect to trusted and confidential information. MiniScope answers whether the agent has only the service permissions needed for the task. A secure agent runtime may need all three kinds of enforcement because each protects a different boundary.

7.2 Implications for Secure Agent Runtime Design

The comparison suggests that future agent platforms should treat the LLM planner as an untrusted or partially trusted component. The planner may remain useful for interpreting natural language, decomposing tasks, and proposing tool calls, but it should not be the final authority over external actions. Instead, the runtime should mediate tool use through explicit mechanisms such as contextual policies, information-flow labels, permission checks,

isolation boundaries, and audit logs.

This resembles traditional operating-system security. Operating systems do not rely on applications to voluntarily avoid unsafe file, memory, or network operations. They enforce boundaries through access control, process isolation, reference monitors, and privilege separation. Tool-calling agents require analogous mechanisms for AI-mediated workflows. The protected resources are different—tools, credentials, memory, private data, and external services—but the design principle is similar: powerful programs should be constrained by system-level enforcement.

A layered agent runtime could therefore combine the three control planes studied in this report. At the authority layer, a MiniScope-like mechanism could isolate credentials and grant only the permissions required for a task. At the information-flow layer, a FIDES-like mechanism could label data, track provenance, and block unsafe influence or egress. At the purpose layer, a Conseca-like mechanism could check whether proposed actions match the current user task and trusted context. Such a runtime would not eliminate all risk, but it would make the security boundary more explicit, inspectable, and enforceable.

7.3 Future Work

Several research directions remain open. First, future work should study how to compose multiple enforcement layers. Conseca, FIDES, and MiniScope each define a local security mechanism, but real deployments may need all of them at once. The runtime must decide how to order checks, resolve conflicts, and explain denials to users.

Second, future work should improve enforcement granularity. Conseca depends on the granularity of trusted context and policy constraints. FIDES depends on the granularity and correctness of labels. MiniScope depends on the scopes exposed by services. Across all three systems, coarse granularity improves deployability but weakens precision, while fine granularity improves security but increases engineering and usability costs.

Third, future work should develop multidimensional benchmarks for agent security. Current evaluations often focus on one dimension, such as prompt-injection success, task completion, or permission minimality. Secure agent evaluation should measure task utility, attack resistance, data-flow safety, permission minimality, runtime overhead, and user burden together.

Finally, future work should extend these ideas to multi-agent and long-running agent systems. Future agents may delegate tasks, share memory, act across organizational boundaries, and maintain persistent state over time. These settings make purpose, information flow, and authority harder to define because user intent, trust boundaries, and permission states may evolve during execution.

7.4 Final Perspective

The evolution of tool-calling agents makes clear that agent security is not only a model-alignment problem. It is also a systems problem. As agents gain access to more tools, private data, external services, and long-running workflows, security must be enforced at the runtime boundary where model reasoning becomes action.

Conseca, FIDES, and MiniScope show three promising directions for this transition. They demonstrate that deterministic enforcement can be added around contextual purpose, information flow, and delegated authority. Their limitations also show that no single mechanism is sufficient. The most plausible path forward is a layered design in which an unreliable but useful LLM planner is surrounded by system-level controls. Secure agentic AI may therefore depend less on making models perfectly trustworthy and more on building runtimes that remain safe even when models are imperfect.

References

[1] Android Developers. Permissions on Android. https://developer.android.com/guide/topics/permissions/overview, 2025. Accessed: 2026-05-05.

[2] Anthropic. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol, November 2024. Accessed: 2026-05-05.

[3] Harrison Chase. Langchain, 2022. URL https://github.com/langchain-ai/langchain. Released on 2022-10-17.

[4] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2383–2400, Seattle, WA, August 2025. USENIX Association. ISBN 978-1-939133-52-6. URL https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe.

[5] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. Secalign: Defending against prompt injection with preference optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, page 2833–2847, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400715259. doi: 10.1145/3719027.3744836. URL https://doi.org/10.1145/3719027.3744836.

[6] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing ai agents with information-flow control, 2025. URL https://arxiv.org/abs/2505.23643.

[7] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=m1YYAQjO3w.

[8] Dorothy E. Denning. A lattice model of secure information flow. *Commun. ACM*, 19 (5):236–243, May 1976. ISSN 0001-0782. doi: 10.1145/360051.360056. URL https://doi.org/10.1145/360051.360056.

[9] Juhee Kim, Xiaoyuan Liu, Zhun Wang, Shi Qiu, Bo Li, Wenbo Guo, and Dawn Song. The attack and defense landscape of agentic ai: A comprehensive survey, 2026. URL https://arxiv.org/abs/2603.11088.

[10] LangChain. LangGraph: Build resilient language agents as graphs. https://github.com/langchain-ai/langgraph, 2025. Accessed: 2026-05-05.

[11] Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Ren Lu, Thomas Mesnard, Johan Ferret, Colton Bishop, Ethan Hall, Victor Carbune, and Abhinav Rastogi. RLAIF: Scaling reinforcement learning from human feedback with AI feedback, 2024. URL https://openreview.net/forum?id=AAxIs3D2ZZ.

[12] Barry Leiba. Oauth web authorization protocol. *IEEE Internet Computing*, 16(1):74–77, January 2012. ISSN 1089-7801. doi: 10.1109/MIC.2012.11. URL https://doi.org/10.1109/MIC.2012.11.

[13] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against llm-integrated applications, 2024. URL https://arxiv.org/abs/2306.05499.

[14] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847, Philadelphia, PA, August 2024. USENIX Association. ISBN 978-1-939133-44-1. URL https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei.

[15] MITRE. MITRE ATLAS: Adversarial threat landscape for artificial-intelligence systems. https://atlas.mitre.org/, 2026. Accessed: 2026-05-05.

[16] OpenAI. Computer-using agent. https://openai.com/index/computer-using-agent/, January 2025. Accessed: 2026-05-05.

[17] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

[18] OWASP Foundation. OWASP Top 10 for Large Language Model Applications 2025. https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf, 2025. Accessed: 2026-05-05.

[19] Alexander Robey, Eric Wong, Hamed Hassani, and George J. Pappas. SmoothLLM: Defending large language models against jailbreaking attacks, 2024. URL https://openreview.net/forum?id=xq7h9nfdY2.

[20] A. Sabelfeld and A.C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003. doi: 10.1109/JSAC.2002.806121.

[21] J.H. Saltzer and M.D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.

[22] Ravi S. Sandhu. Role-based access control11portions of this chapter have been published earlier in sandhu et al. (1996), sandhu (1996), sandhu and bhamidipati (1997), sandhu et al. (1997) and sandhu and feinstein (1994). volume 46 of *Advances in Computers*, pages 237–286. Elsevier, 1998. doi: https://doi.org/10.1016/S0065-2458(08)60206-5. URL https://www.sciencedirect.com/science/article/pii/S0065245808602065.

[23] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf.

[24] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for llm agents, 2025. URL https://arxiv.org/abs/2504.11703.

[25] Lillian Tsai and Eugene Bagdasarian. Contextual agent security: A policy for every purpose. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, HOTOS '25, page 8–17. ACM, May 2025. doi: 10.1145/3713082.3730378. URL http://dx.doi.org/10.1145/3713082.3730378.

[26] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents, 2025. URL https://arxiv.org/abs/2503.18666.

[27] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: how does llm safety training fail? In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, 2023. Curran Associates Inc.

[28] Simon Willison. The dual LLM pattern for building AI assistants that can resist prompt injection. https://simonwillison.net/2023/Apr/25/dual-llm-pattern/, April 2023. Accessed: 2026-05-05.

[29] Simon Willison. The lethal trifecta for AI agents: Private data, untrusted content, and external communication. https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/, June 2025. Accessed: 2026-05-05.

[30] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Proceedings of the Network and Distributed System Security Symposium*, NDSS 2025, San Diego, CA, USA, February 2025. Internet Society. ISBN 979-8-9894372-8-3. doi: 10.14722/ndss.2025.241131. URL https://www.ndss-symposium.org/ndss-paper/isolategpt-an-execution-isolation-architecture-for-llm-based-agentic-systems/.

[31] Zhangchen Xu, Fengqing Jiang, Luyao Niu, Jinyuan Jia, Bill Yuchen Lin, and Radha Poovendran. SafeDecoding: Defending against jailbreak attacks via safety-aware decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024. URL https://aclanthology.org/2024.acl-long.303/.

[32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[33] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In Lun-Wei

Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.624. URL https://aclanthology.org/2024.findings-acl.624/.

[34] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G. Patil, Vivian Fang, and Raluca Ada Popa. Miniscope: A least privilege framework for authorizing tool calling agents, 2025. URL https://arxiv.org/abs/2512.11147.